SINAV CEVAP KÂĞIDI

Öğrenci tarafından imzalanmayan sınav kâğıdı geçersiz sayılır.

T.C.

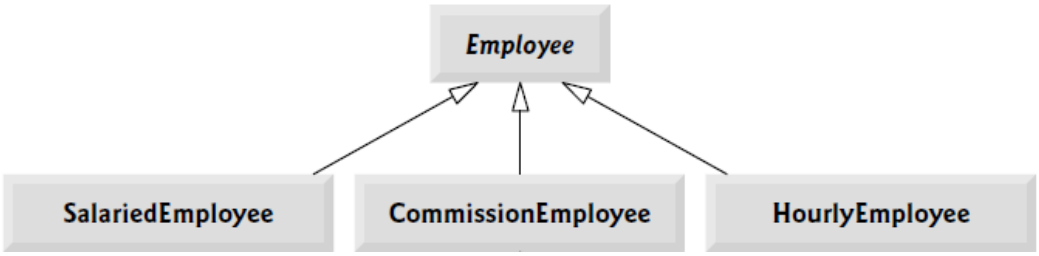
BOZOK ÜNİVERSİTESİ

Mühendislik- Mimarlık Fakültesi

Adı ve Soyadı		
No ve İmza		

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ 2024-2025 GÜZ DÖNEMİ FİNAL SINAVI

Dersin Adı : NESNEYE DAYALI PROGRAMLAMA	Soru No	Toplam
Kod No : Tarih : 09.01.2025	NOT	



Employee hiyerarşisi yukarıda verilmiştir. Abstract bir sınıf olan Employee sınıfı yukarıda verilen şekilde görüldüğü gibi bir hiyerarşi oluşturur. Her bir çalışan ister maaşlı (salariedemployee), ister komisyonlu(comissionemployee), isterse saatlik çalışan olsun (hourlyemployee), adı (name), soyadı(surname) ve sosyal güvenlik numarası(scn) olmalıdır. Yani bu alanlar (field) Employee sınıfı içinde tanımlanmalıdır. Employee sınıfı Earnings (Kazanc için) ve ToString() metotları barındırmaktadır. Çalışanın adı, soyadı ve sosyal güvenlik numarası yapıcı ile alınmalıdır. Bu bilgiler istenildiğinde salt okunur propertyler kullanılarak görülmelidir. Ayrıca Çalışanların kazançlarının kendi sınıflarına göre hesaplanmasını zorunlu kılacak bir abstract metot Earning() olmalıdır. Her bir türetilen sınıf kendine göre bu Earning() metodunu kendine göre hesaplamalıdır. ToString() metodu ise Çalışanın adı, soyadı ve sosyal güvenlik numarasını görüntülemek için kullanılmalıdır. Ancak türetilen sınıflarda bu metot aşağıda verilen şekilde görüldüğü üzere diğer alanların görüntülenmesini sağlamak için kullanılmalıdır. görüldüğü üzere Salaried employee sınıfı Employee sınıfından türemektedir. Employee sınıfına ek olarak kurucusunda weeklysalary değerini (haftalık maaş tutulması için) yapıcısında almalıdır. Earning metodu çalışanın yapıcı ile girilen haftalık maaş değerini return etmelidir. Bu sınıf içindeki ToString() metodunun bildirmesi gereken değerler aşağıda görülmektedir.

HourlyEmployee (saatlik ücretle çalışanların alacağı ücretlerin tespiti için oluşturulan sınıf) bu sınıf Employee sınıfına ek olarak kurucu içinde wage (ucret) ve hours(haftalık toplam çalışma süresi için) değişkenlerini almalı Wage property si wage değişkeninin negatif olmamasını garantiye almalıdır. Ayrıca Hours property si kullanılarak hours değerinin 0 ile 168 arasında olması sağlanmalıdır.(haftalık çalışma saati için) Earning metodu şekilde görüldüğü gibi 40 saatten az ve çok olmak üzere iki türlü hesap yapmalıdır. Eğer bu değerler arasında veri girişi yapılmamış ise argumentoutofrange exception fırlatmalıdır. ToStringMetodu ise aşağıda belirtilen değişkenler hakkında bilgileri içermelidir.

CommissionEmployee Sınıfı görüldüğü üzere Employee sınıfından türemekte olup yapıcısı içerisinde Employee sınıfında istenenlere ek olarak satış miktarı ve komisyon oranı değişkenlerinin tutulması için commissionRate ve grossSales değerlerini almalıdır. Earnings() metodu bu iki değerin çarpımını döndürmelidir. ToString metodu ise aşağıda belirtilen değerler hakkında bilgi vermelidir.

Bu açıklamaları kullanarak aşağıdaki test sorularını cevaplayınız.

İstenilen açıklamaları temel alarak her birini 5 şıklı test sorusuna dönüştürdüm. Sorular, verilen bilgileri ve hiyerarşiyi anlayıp uygulamaya yönelik olarak hazırlanmıştır.

1-9 ARASI SORULARI BU AÇIKLAMA VE TABLOYA GÖRE CEVAPLAYINIZ.

S1. Employee** sınıfıyla ilgili aşağıdaki ifadelerden hangisi yanlıştır?
a) Employee sınıfı bir abstract sınıftır. b) Employee sınıfı, Name, Surname ve SocialSecurityNumber özelliklerini içerir. c) Employee sınıfının Earnings() metodu implement edilmelidir. d) Employee sınıfından doğrudan nesne türetilebilir. e) ToString() metodu çalışan bilgilerini döndürür.
Cevap:d
S2. SalariedEmployee** sınıfında aşağıdaki özelliklerden hangisi bulunmaz?
a) WeeklySalary özelliği. b) Name ve Surname özellikleri. c) Earnings() metodu.
d) CommissionRate özelliği. e) ToString() metodu.
Cevap:d
S3. HourlyEmployee** sınıfındaki `Hours` özelliği için aşağıdaki koşullardan hangisi doğrudur?
a) `Hours` değeri negatif olabilir. b) `Hours` değeri 0 ile 168 arasında olmalıdır.
c) `Hours` özelliği `string` olarak tanımlanmalıdır. d) `Hours` özelliği, kullanıcıdan alınan tüm değerleri kabul eder. e) `Hours` değerine herhangi bir kısıtlama uygulanmaz.
Cevap:b
S4. Aşağıdaki kod parçası için hangi hata alınır?

```
public abstract class Employee
{
  public abstract decimal Earnings();
}
class Program
{
  static void Main()
  {
    Employee emp = new Employee();
  }
}
```

a) Runtime error: `Employee` sınıfının eksik bir metodu vardır.
b) Compile-time error: `Earnings` metodu implement edilmelidir.
c) Compile-time error: Abstract sınıfların instance'ı alınamaz.
d) Compile-time error: `Employee` bir arayüz olmalıdır.
e) Hiçbiri.
Doğru Cevap: c
S5. **CommissionEmployee** sınıfında `CommissionRate` ile ilgili doğru ifade hangisidir?
a) `CommissionRate` negatif bir değer alabilir.
b) `CommissionRate` bir property olarak tanımlanamaz.
c) `CommissionRate`, sadece getter ile tanımlanmalıdır.
d) `CommissionRate`, sabit bir değerdir.
e) `CommissionRate`, 0 ile 1 arasında bir değer olmalıdır.
Cevap: e
S6. Aşağıdaki `HourlyEmployee` sınıfında fazla mesai (40 saatten fazla) nasıl hesaplanır?
a) Fazla mesai, saatlik ücretle çarpılır.
b) Fazla mesai ücreti, normal çalışma saatlerine eklenir.
c) Fazla mesai için standart ücret uygulanır.
d) 40 saatin üzerindeki çalışma saatleri 1.5 kat ücretle hesaplanır.
e) Fazla mesai ücreti hesaplanmaz.
Cevap: d
S7. IPayable arayüzü aşağıdaki yöntemlerden hangisini içermelidir?
a) public string GetPaymentDetails(); b) public decimal GetPaymentAmount();
c) public void CalculatePayment(); d) public bool ValidatePayment();
e) public int CalculateAmount(); Cevap:b

S8. Aşağıdaki sınıflardan hangisi `Employee` sınıfından türetilmemiştir?
a) SalariedEmployee
b) HourlyEmployee
c) CommissionEmployee
d) IPayable
e) Hepsi `Employee` sınıfından türetilmiştir.
Cevap:** d

S9. Aşağıdaki `SalariedEmployee` sınıfında `ToString` metodu için hangi çıktı alınır?

```csharp
SalariedEmployee emp = new SalariedEmployee("Ali", "Yılmaz", "123", 2000);
Console.WriteLine(emp.ToString());
```

a) Ali Yılmaz, SSN: 123, Weekly Salary: 2000
b) Ali Yılmaz earns 2000 weekly.
c) Ali Yılmaz works 40 hours.
d) Compile-time error.
e) Hiçbiri.
Cevap: a

S10. Aşağıdakilerden hangisi bir C# Exception olarak değerlendirilmez?
a. Kullanıcıdan sayı istenir, ancak bir karakter girer.
b. Bir değişken türü için çok büyük bir değer hesaplanır.
c. Çok büyük bir indis değeriyle bir diziye erişim yapılır.
d. Bir C# programını çalıştırmaya çalışırsınız, ancak C# derleyicisi kurulu değildir.
e. Kullanıcıdan veri alınır ancak programda işlenmez.
Doğru cevap: d

S11. Bir try bloğu ne için kullanılır?
a. Hataları loglamak
b. Bellek yönetimi sağlamak
c. İstisnaları yeniden fırlatmak
d. Kodları yok saymak
e. Potansiyel istisnaların oluşabileceği kodları sarmalamak
Doğru cevap: e

S12. Bir istisna oluştuğunda aşağıdaki bloklardan hangisi her zaman çalıştırılır?
a. try
b. catch
c. finally
d. throw
e. log
Doğru cevap: c

S13. catch bloğu ne zaman çalıştırılır?
a. try bloğu istisna fırlattığında
b. try bloğu başarılı bir şekilde tamamlandığında
c. finally bloğu çalıştırıldığında
d. Program başlatıldığında
e. Hiçbiri
Doğru cevap: a

S14. Bir Exception fırlatmak için kullanılan anahtar kelime nedir?
a. Raise b. Try c. Catch d. throw e. emit
Doğru cevap: d

S15. Aşağıdaki hangi Exception tüm istisnaları yakalar?
a. NullReferenceException b. DivideByZeroException
c. Exception d. IndexOutOfRangeException
e. ArgumentException
Doğru cevap: c
16. Aşağıdaki kodda, CertifiedLetter sınıfının ToString() metodu temel sınıfın metoduna nasıl erişir?

```
class CertifiedLetter : Letter
{
  public string TrackingNumber { get; set; }

  public override string ToString()
  {
    return base.ToString() + $", Tracking Number = {TrackingNumber}";
  }
}
```

a. base.ToString() kullanarak b. this.ToString() kullanarak
c. super.ToString() kullanarak d. Doğrudan çağrı ile
e. Bu işlem yapılamaz
Doğru cevap: a
S17. Aşağıdaki tanım hangi OOP prensibine aittir?
"Bir alt sınıf, üst sınıfın metotlarının veya özelliklerinin yeniden tanımlanmasına izin verir."
a. Kapsülleme (Encapsulation) b. Kalıtım (Inheritance)
c. Polimorfizm (Polymorphism) d. Soyutlama (Abstraction) e. Hiçbiri
Cevap C
S18. "Bir sınıfın diğer sınıflardan gizlenmiş ayrıntılara sahip olması, ancak o sınıfa erişim için belirli metotlar sunması" hangi prensibi ifade eder?
a. Polimorfizm b. Kapsülleme c. Kalıtım d. Soyutlama
e. Dependency Injection
Cevap: b
19. Bir sınıfta "base" anahtar kelimesi ne için kullanılır?
a. Alt sınıftan üst sınıfın metotlarına erişim için
b. Aynı sınıftaki diğer metotlara erişim için
c. Sanallaştırılmış metotları tanımlamak için
d. Soyut metotları çağrırken e. Hiçbiri
Cevap:a
20. Aşağıdaki kodun çıktısı nedir?

```
int number = 5;
switch (number)
{
  case 1:    Console.WriteLine("One");    break;  case 5:  Console.WriteLine("Five");
    break;
  default:
    Console.WriteLine("Other");
    break;
}
```

a. Compile error b. Five c. Other d. One e. Hiçbiri
Cevap:** b
21. Aşağıdaki kodun çıktısı nedir?

```
int[] numbers = { 1, 2, 3, 4, 5 };
Console.WriteLine(numbers[2]);
```

a. 2 b. 3 c. 4 d. Compile error e. Hiçbiri
Cevap:b

22. Bir dizinin tüm elemanlarını tersine çevirmek için hangi metot kullanılır?
a. Reverse() b. Sort() c. Length d. Clear() e. Add()
Cevap:** a
23.Aşağıdaki kod parçası hangi hatayı verir?
```
int[] array = new int[3]; array[5] = 10;```
a. Compile error b. NullReferenceException c. ArgumentException
d. IndexOutOfRangeException e. OverflowException
Cevap:** d
**24.** "List" koleksiyonunda yeni bir eleman eklemek için hangi metot kullanılır?
a. Add() b. Insert() c. Append() d. Push() e. Hiçbiri
Cevap:** a
**25.** Bir dizínin boyutunu almak için hangi özellik kullanılır?
a. Count b. Size c. Length d. GetLength() e. Hiçbiri
Cevap:c
**26.** Aşağıdaki kod parçası hangi hatayı verir?
```csharp
Dictionary<string, int> scores = new Dictionary<string, int>();
Console.WriteLine(scores["Alice"]);
```
a. OverflowException b. NullReferenceException c. ArgumentException
d. Compile error e. KeyNotFoundException
Cevap:e
27. Aşağıdaki kodun çıktısı nedir?
```
abstract class Base
{
  public abstract void AbstractMethod();
  public virtual void VirtualMethod() => Console.WriteLine("Base Virtual Method");
}
class Derived : Base
{
  public override void AbstractMethod() => Console.WriteLine("Derived Abstract Method");
  public override void VirtualMethod() => Console.WriteLine("Derived Virtual Method");
}
--main kısmı--
Base obj = new Derived();
obj.AbstractMethod();
obj.VirtualMethod();
```
A) Derived Abstract Method
B) Base Virtual Method
C) Kod çalışmaz, çünkü abstract sınıfın virtual metotları olamaz.
D) Derived Abstract Method ve ardından Derived Virtual Method
E) Kod çalışmaz, çünkü VirtualMethod override edilmemiştir.
Doğru Cevap: d
28. Çoklu Interface Kullanımı
Aşağıdaki kodun çıktısı nedir?
```
interface IFirst
{
  void Display();
}
interface ISecond
{
```
```

```
  void Display();
}

class MultiInterface : IFirst, ISecond
{
  void IFirst.Display() => Console.WriteLine("First Interface Display");
  void ISecond.Display() => Console.WriteLine("Second Interface Display");
}
IFirst first = new MultiInterface();
ISecond second = new MultiInterface();
first.Display();
second.Display();
```
A) First Interface Display ve ardından Second Interface Display
B) Second Interface Display
C) Kod çalışmaz, çünkü aynı isimli metotlar bir sınıfta implement edilemez.
D) Kod çalışmaz, çünkü explicit interface implementasyonu eksik.
E) Kod çalışmaz, çünkü interface'de metotlara default implementasyon atanmış olmalıdır.
Doğru Cevap: A
29. Aşağıdaki kod çalıştırıldığında ne olur?
```
class BaseClass
{
  public virtual void Display() => Console.WriteLine("BaseClass Display");
}

class DerivedClass : BaseClass
{
  public sealed override void Display() => Console.WriteLine("DerivedClass Display");
}

class SubDerivedClass : DerivedClass
{
  public override void Display() => Console.WriteLine("SubDerivedClass Display");
}
```
A) SubDerivedClass Display yazdırılır.
B) DerivedClass Display yazdırılır.
C) Kod çalışmaz, çünkü sealed metot override edilemez.
D) Kod çalışmaz, çünkü sealed yalnızca abstract metotlar için kullanılabilir.
E) Kod çalışmaz, çünkü BaseClass içinde bir hata var.
Doğru Cevap: C
30. Aşağıdaki kod çalıştırıldığında hangi çıktı oluşur?
```
interface IPayable
{
  void Pay();
}

class Invoice : IPayable
{
  public void Pay() => Console.WriteLine("Paying Invoice");
}

class Employee : IPayable
{
```

```
    public void Pay() => Console.WriteLine("Paying Employee");
}

void ProcessPayment(IPayable payable)
{
    payable.Pay();
}

IPayable payable = new Invoice();
ProcessPayment(payable);

payable = new Employee();
ProcessPayment(payable);
```

A) Paying Invoice ve ardından Paying Employee
B) Paying Employee
C) Paying Invoice
D) Kod çalışmaz, çünkü interface ile method parametresi kullanılmaz.
E) Kod çalışmaz, çünkü ProcessPayment metodu bir interface parametresini kabul edemez.
Doğru Cevap: a

31. Aşağıdaki kodun çıktısı nedir?

```
abstract class Shape
{
    public abstract void Draw();
    public void Display() => Console.WriteLine("Displaying Shape");
}

class Circle : Shape
{
    public override void Draw() => Console.WriteLine("Drawing Circle");
}
--main--
Shape shape = new Circle();
shape.Draw();
shape.Display();
```

A) Displaying Shape
B) Drawing Circle
C) Kod çalışmaz, çünkü Display metodu override edilmemiştir.
D) Kod çalışmaz, çünkü abstract sınıftan nesne oluşturulamaz.
E) Drawing Circle ve ardından Displaying Shape
Doğru Cevap: e

32. Aşağıdaki kodun çıktısı nedir?

```
interface IDevice
{
    void Start();
    void Stop() => Console.WriteLine("Stopping Device");
}
class Printer : IDevice
{
    public void Start() => Console.WriteLine("Starting Printer");
```

```
}
--main--
IDevice device = new Printer();
device.Start();
device.Stop();
```
A) Starting Printer B) Stopping Device C) Kod çalışmaz, çünkü interface'de default metot tanımlanamaz. D) Starting Printer ve ardından Stopping Device
E) Kod çalışmaz, çünkü Printer sınıfı Stop metodunu implement etmemiştir.
Doğru Cevap: d

33. Aşağıdaki kod çalıştırıldığında ne olur?
```
sealed class Utility
{
   public void PerformTask() => Console.WriteLine("Performing Utility Task");
}
class ExtendedUtility : Utility
{
   public void PerformTask() => Console.WriteLine("Performing Extended Task");
}
```
A) Performing Utility Task
B) Kod çalışmaz, çünkü Utility sealed olduğu için türetilemez.
C) Performing Extended Task
D) Kod çalışmaz, çünkü sealed sınıf yalnızca abstract sınıf olabilir.
E) Kod çalışmaz, çünkü PerformTask metodu override edilmemiştir.
Doğru Cevap: B

34.Aşağıdaki kodun çıktısı nedir?
```
interface ILogger
{
   void Log(string message);
   void Error(string error) => Console.WriteLine($"Error: {error}");
}
class ConsoleLogger : ILogger
{
   public void Log(string message) => Console.WriteLine($"Log: {message}");
}
ILogger logger = new ConsoleLogger();
logger.Log("Hello, World!");
logger.Error("Something went wrong.");
```
A) Log: Hello, World!
B) Error: Something went wrong.
C) Log: Hello, World! ve ardından Error: Something went wrong.
D) Kod çalışmaz, çünkü interface metotlarına varsayılan uygulama atanamaz.
E) Kod çalışmaz, çünkü ConsoleLogger sınıfı tüm metotları implement etmelidir.
Doğru Cevap: C

35. `Action` delegate** neyi temsil eder?
a) Parametre almayan ve geri dönüş değeri olmayan metotları.
b) Geri dönüş değeri `bool` olan metotları.
c) Bir veya daha fazla parametre alabilen ve geri dönüş değeri olan metotları.
d) Asenkron olarak çalışan metotları.
e) Geri dönüş değeri olmayan, bir veya daha fazla parametre alabilen metotları.
Cevap:e

36. Aşağıdaki kodun çıktısı nedir?

```csharp
class Program
{
  static void Main(string[] args)
  {
    Action<string, int> printData = (name, age) =>
    {
      Console.WriteLine($"{name} is {age} years old.");
    };

    printData("Ali", 25);
  }
}
```
a) `Compile-time error` b) `Ali is 25 years old.` c) `Runtime error`
d) `25 is Ali years old.` e) Hiçbiri.
Cevap:b
37. Task`** sınıfı ile ilgili hangisi yanlıştır?
a) `Task`, bir işi asenkron olarak çalıştırmak için kullanılır.
b) `Task` sınıfı, bir delegate'i çalıştırabilir.
c) `Task` sınıfının bir iş tamamlandığında sonuç döndürebilmesi için `Task<T>` türü kullanılmalıdır.
d) `Task` nesnesi, birden fazla iş için paralel çalışmayı destekler.
e) `Task` yalnızca senkron işlemleri çalıştırabilir.
Doğru Cevap: e
38. Aşağıdaki kod çalıştırıldığında hangi çıktı alınır?

```csharp
using System;
using System.Threading.Tasks;

class Program
{
  static async Task Main(string[] args)
  {
    Task<int> task = Task.Run(() =>
    {
      return 42;
    });

    int result = await task;
    Console.WriteLine($"Result: {result}");
  }
}
```
a) `Result: 42` b) `Compile-time error` c) `Runtime error`
d) `Result: 0` e) Hiçbiri

Doğru Cevap: a

39. Aşağıdaki kodda `Action` ve `Task` birlikte kullanıldığında hangi ifade doğrudur?
```csharp
class Program
```

```
{
  static void Main(string[] args)
  {
    Action printMessage = () =>
    {
      Console.WriteLine("Task started!");
    };

    Task task = Task.Run(() => printMessage());
    task.Wait();
  }
}
```
a) Kod hata verir çünkü `Action` delegate bir `Task` içinde kullanılamaz.
b) `Task started!` çıktısı ekrana yazılır.
c) Kod hiçbir şey yazdırmaz çünkü `Action` delegate boş bir metot içerir.
d) Kod asenkron olarak çalışır ve çıktıyı yazdırmadan tamamlanır.
e) Kod derlenemez çünkü `Task.Wait()` metodu eksiktir.
cevap:b
40. Aşağıdaki kodun çıktısı nedir?
```csharp
using System;
using System.Collections.Generic;
class Program
{
  static void Main()
  {
    List<int> numbers = new List<int> { 10, 15, 20, 25, 30 };
    Predicate<int> isGreaterThan20 = (n) => n > 20;

    int firstMatch = numbers.Find(isGreaterThan20);
    Console.WriteLine(firstMatch);
  }
}
```
a) `10` b) `15` c) `20` d) `25` e) `Compile-time error`

Doğru Cevap: d

41. Aşağıdaki kodda hangi elemanlar listeden kaldırılır?
```csharp
using System;
using System.Collections.Generic;

class Program
{
  static void Main()
  {
    List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6 };
    Predicate<int> isOdd = (n) => n % 2 != 0;
    numbers.RemoveAll(isOdd);
    foreach (var num in numbers)
```

```
    {
        Console.WriteLine(num);
    }  }}```
```
a) Tüm çift sayılar kaldırılır. b) Tüm tek sayılar kaldırılır. c) Liste boş kalır.
d) Liste değiştirilmez. e) `Compile-time error`
Cevap:** b

42.Aşağıdaki kodun çıktısı nedir?
```
class Circle
{
  public readonly double Radius;

  public Circle(double radius)
  {
    Radius = radius;
  }
}
Circle c = new Circle(5);
c.Radius = 10;
```
A) Radius 10 olarak değişir. B) Radius 5 olarak kalır, ama hata vermez.
C) Kod çalışmaz, çünkü sınıfta bir hata var. D) Kod çalışmaz, çünkü constructor içinde Radius set edilmelidir. E) Kod çalışmaz, çünkü readonly bir özellik değiştirilemez.
Cevap: e

43. Aşağıdaki kodun çıktısı nedir?
```
struct Complex
{
  public double Real { get; set; }
  public double Imaginary { get; set; }

  public static Complex operator +(Complex c1, Complex c2)
  {
    return new Complex
    {
      Real = c1.Real + c2.Real,
      Imaginary = c1.Imaginary + c2.Imaginary
    };  }}
Complex c1 = new Complex { Real = 1, Imaginary = 2 };
Complex c2 = new Complex { Real = 3, Imaginary = 4 };
Complex result = c1 + c2;
Console.WriteLine($"{result.Real}, {result.Imaginary}");
```
A) 4, 6 B) 1, 2 C) 3, 4
D) Kod çalışmaz, çünkü struct içinde operatör aşırı yüklenemez.
E) Kod çalışmaz, çünkü Complex için bir constructor eksik.
Doğru Cevap: A
44. Aşağıdaki kodun çıktısı nedir?
```
class Employee
{
  public string Name { get; set; }

  public Employee(string name)
  {
```

```
    this.Name = name;
  }}
Employee e = new Employee("John");
Console.WriteLine(e.Name);
```

A) null B) John C) Kod çalışmaz, çünkü this kullanımı yanlış.
D) Kod çalışmaz, çünkü bir default constructor eksik.
E) Kod çalışmaz, çünkü Name readonly olmalıdır.
Doğru Cevap: b

45. Aşağıdaki kodun çıktısı nedir?

csharp
Kodu kopyala

```
public delegate int Calculate(int a, int b);
class Program
{
  public static void Main(string[] args)
  {
    Calculate calc = (a, b) => a + b;
    calc += (a, b) => a * b;
    Console.Write(calc.Invoke(3, 4));
  }}
```

a) 7 b) 12 c) 712 d) 127 e) Compile-time error

Doğru Cevap: e

46. Destructor hangi amaçla kullanılır?
A) Static özellikleri initialize etmek için.
B) Bellekteki kaynakları serbest bırakmak için.
C) Sınıfların static olmayan özelliklerini temizlemek için.
D) Default constructor yerine kullanılır.
E) readonly özellikleri değiştirmek için.
Doğru Cevap: B

47. Aşağıdaki kodun çıktısı nedir?

```
class Data
{
  public static readonly int MaxValue = 100;
  static Data()
  {
    MaxValue = 200;
  }
}
```

A) Kod çalışır ve MaxValue 200 olur.
B) Kod çalışır ve MaxValue 100 olur.
C) Kod çalışmaz, çünkü readonly static constructor içinde değiştirilemez.
D) Kod çalışmaz, çünkü readonly bir özellik set edilmiyor.
E) Kod çalışmaz, çünkü static constructor içinde hata var.
Doğru Cevap: A

48. Aşağıdaki kodun çıktısı nedir?

```
class Student
{
  public string Name { get; set; }
  public int Age { get; set; }
}
```

```
Student s = new Student { Name = "Alice", Age = 20 };
Console.WriteLine($"{s.Name}, {s.Age}");
```
A) null, 20
B) Alice, 0
C) Alice, 20
D) Kod çalışmaz, çünkü initializer yalnızca struct için geçerlidir.
E) Kod çalışmaz, çünkü özelliklere varsayılan değer atanmalıdır.
Cevap: C
49.readonly bir özellik ne zaman set edilebilir?
A) Yalnızca static metot içinde. B) Yalnızca constructor içinde.
C) Sınıf dışında her zaman. D) Yalnızca bir static constructor içinde.
E) Hiçbir zaman değiştirilemez.
Doğru Cevap: B
50.Aşağıdaki kodun çıktısı nedir?
```
struct Point
{
  public int X { get; set; }
  public int Y { get; set; }

  public static bool operator ==(Point p1, Point p2)
  {
    return p1.X == p2.X && p1.Y == p2.Y;
  }

  public static bool operator !=(Point p1, Point p2)
  {
    return !(p1 == p2);
  }
}

Point p1 = new Point { X = 1, Y = 2 };
Point p2 = new Point { X = 1, Y = 2 };
Console.WriteLine(p1 == p2);
```
A) True
B) False
C) Kod çalışmaz, çünkü struct üzerinde operatör aşırı yüklenemez.
D) Kod çalışmaz, çünkü struct için Equals ve GetHashCode override edilmelidir.
E) Kod çalışmaz, çünkü == ve != operatörleri birlikte tanımlanmalıdır.
Doğru Cevap: d
51. Aşağıdaki kodun çıktısı nedir?
```
class Configuration
{
  public static string Setting { get; private set; }

  static Configuration()
  {
    Setting = "Default";
  }
}

Console.WriteLine(Configuration.Setting);
```

A) Default
B) null
C) Kod çalışmaz, çünkü static constructor içinde bir özellik set edilemez.
D) Kod çalışmaz, çünkü Setting erişimci eksik.
E) Kod çalışmaz, çünkü sınıf bir nesne olmadan kullanılamaz.
Doğru Cevap: A

52. Aşağıdakilerden hangisi .NET tür hiyerarşisinin köküdür?
A. System.Parent B. Sistem.Type C. System.Base
D. System.Object E. System.Root
Cevap d
53.Aşağıdaki kodun çıktısı nedir?

```
class Account
{
   public static double InterestRate { get; set; } = 0.05;
   public double Balance { get; set; }
}

Account a = new Account { Balance = 1000 };
Console.WriteLine(a.Balance * Account.InterestRate);
```

A) 0.05 B) 50 C) 1000
D) Kod çalışmaz, çünkü static özellik object initializer ile kullanılamaz.
E) Kod çalışmaz, çünkü InterestRate yalnızca readonly olmalıdır.
Cevap: b

54. Private constructor neden kullanılır?
A) Sınıfı başka sınıflardan türetmek için.
B) Sınıfı yalnızca struct ile kullanılabilir yapmak için.
C) Sınıfın static özelliklerini değiştirmek için.
D) Sınıfın yalnızca bir static metotla kullanılmasını sağlamak için.
E) Constructor'ı override etmek için.
Cevap: d

55. Aşağıdaki kodun çıktısı nedir?

```
class Example
{
   public static int StaticValue = 10;
   public int InstanceValue = 20;
}

Example ex1 = new Example();
Example ex2 = new Example();
ex1.InstanceValue = 30;
ex2.InstanceValue = 40;
Console.WriteLine(Example.StaticValue);
```

A) 10 B) 20 C) 30 D) 40
E) Kod çalışmaz, çünkü static özellikler instance üzerinden çağrılamaz.
Doğru Cevap: A
56. Aşağıdaki metod ref parametresini nasıl kullanır?

```
private static void UpdateValue(ref int value)
{
   value += 10;
```

}
Metod çağrısı aşağıdaki gibi yapılırsa, x'in değeri nedir?
int x = 5;
UpdateValue(ref x);
A) 5 B) 10 C) 15 D) 20 E)hiçbiri
Cevap: C
56. Bir metod çağrısında neden "ambiguous" hatası alınır?
A) İki metod aynı parametre listesine sahiptir.
B) İki metod aynı dönüş türüne sahiptir.
C) İki metod aynı isimde ancak farklı sınıflardadır.
D) Parametre listesi, overload çözümüne izin vermez.
Cevap: A

57. params anahtar kelimesi ne için kullanılır?
A) Opsiyonel parametreleri tanımlamak için.
B) Dinamik veri tiplerini işlemek için.
C) Referans parametreleri işlemek için.
D) Metodun bir dizi uzunluktaki argümanları kabul etmesini sağlamak için.
E)hiçbiri
Cevap: e
58.Aşağıdaki kod çalıştırıldığında ne olur?

```
class Example
{
  int instanceValue = 10;

  public static void DisplayValue()
  {
    Console.WriteLine(instanceValue);
  }
}
Example.DisplayValue();
```

A) 10 yazdırır.
B) Kod çalışmaz, çünkü static bir metotta instance üyeler kullanılamaz.
C) Kod çalışır, ancak instanceValue null olur.
D) Kod çalışmaz, çünkü static metot bir nesne gerektirir.
Cevap: B
59. Aşağıdaki kodda ExampleMethod'a erişmek için ne yapılmalıdır?

```
class Example
{
  private void ExampleMethod()
  {
    Console.WriteLine("Private Method");
  }
}
```

A) ExampleMethod doğrudan çağrılabilir.
B) ExampleMethod sadece sınıfın içinden çağrılabilir.
C) ExampleMethod sadece başka bir sınıfta public olarak tanımlandığında çağrılabilir.
D) Kod çalışmaz, çünkü private metodlar oluşturulamaz.
Doğru Cevap: B
60. BinarySearch() ve Sort() metotlarının ortak özelliği nedir?
A) Her ikisi de tek bir argüman alır ve bu argüman bir dizi olmalıdır.
B) Her ikisi de yalnızca sayısal veri içeren dizilerde çalışır.

C) Dizi artan sırada başlamalıdır.
D) Her ikisi de System.Array sınıfına aittir.
E) Her ikisi de diziyi otomatik olarak sıralar.
Doğru Cevap: D

Ch07_SaleCo Veritabanı Tablo Yapısı

1. CUSTOMER

Cust_ID	Cust_Name	Cust_Balance
1001	John Doe	450.00
1002	Jane Smith	1200.00
1003	Alice White	600.00

2. VENDOR

Vend_ID	Vend_Name	Vend_City
V001	ACME Corp.	New York
V002	Tech Supplies	Chicago
V003	Global Tools	San Francisco

3. INVOICE

Invoice_ID	Cust_ID	Invoice_Total	Invoice_Date
INV001	1001	200.00	2023-01-15
INV002	1002	300.00	2023-02-12
INV003	1003	500.00	2023-03-05

4. LINE

Line_ID	Invoice_ID	Prod_ID	Line_Qty	Line_Price
L001	INV001	P001	2	50.00
L002	INV002	P002	3	100.00
L003	INV003	P003	1	500.00

5. PRODUCT

Prod_ID	Prod_Name	Prod_Price	Stock_Qty
P001	Widget A	50.00	100
P002	Widget B	100.00	50
P003	Widget C	500.00	10

61. Her müşteri için yaptığı toplam satın alım miktarını hesaplayan doğru ifade hangisidir?
A) invoices.GroupBy(i => i.Cust_ID).Select(g => new { Customer = g.Key, Total = g.Sum(i => i.Invoice_Total) });
B) customers.Join(invoices, c => c.Cust_ID, i => i.Cust_ID, (c, i) => new { c.Cust_ID, i.Invoice_Total }).GroupBy(i => i.Cust_ID).OrderByDescending(g => g.Sum(i => i.Invoice_Total));
C) customers.Select(c => invoices.Where(i => i.Cust_ID == c.Cust_ID).Sum(i => i.Invoice_Total));
D) customers.GroupJoin(invoices, c => c.Cust_ID, i => i.Cust_ID, (c, i) => new { c.Cust_Name, Total = i.Sum(inv => inv.Invoice_Total) });
E) invoices.Select(i => new { i.Cust_ID, i.Invoice_Total }).GroupBy(i => i.Cust_ID).Sum();
Cevap: D

62. Müşteriler tarafından yapılan toplam satın alımların alt toplamını hesaplamak için doğru ifade hangisidir?
A) lines.GroupBy(l => l.Invoice_ID).SelectMany(g => g).Sum(l => l.Line_Qty * l.Line_Price);
B) lines.GroupBy(l => l.Invoice_ID).Select(g => new { Invoice = g.Key, SubTotal = g.Sum(l => l.Line_Qty * l.Line_Price) });

C) lines.Select(l => new { l.Invoice_ID, SubTotal = l.Line_Qty * l.Line_Price }).GroupBy(l => l.Invoice_ID).Sum(l => l.SubTotal);
D) invoices.Join(lines, i => i.Invoice_ID, l => l.Invoice_ID, (i, l) => l).Sum(l => l.Line_Qty * l.Line_Price);
E) invoices.Join(lines, i => i.Invoice_ID, l => l.Invoice_ID, (i, l) => new { i.Cust_ID, SubTotal = l.Line_Qty * l.Line_Price }).GroupBy(i => i.Cust_ID).Select(g => new { g.Key, Total = g.Sum(i => i.SubTotal) });
Cevap: E

63. Stokta kalan her ürünün toplam değerini bulmak için doğru ifade hangisidir?
A) products.Select(p => new { p.Prod_Name, TotalValue = p.Stock_Qty * p.Prod_Price }).ToList();
B) products.Where(p => p.Stock_Qty > 0).Sum(p => p.Stock_Qty * p.Prod_Price);
C) products.Select(p => p.Stock_Qty * p.Prod_Price).Sum();
D) products.Aggregate(0.0, (total, p) => total + (p.Stock_Qty * p.Prod_Price));
E) products.Sum(p => p.Stock_Qty * p.Prod_Price);
Cevap: E
64. En çok stok miktarına sahip olan ürünü bulmak için doğru ifade hangisidir?
A) products.Select(p => p.Stock_Qty).OrderByDescending(q => q).First();
B) products.OrderByDescending(p => p.Stock_Qty).FirstOrDefault();
C) products.Where(p => p.Stock_Qty > 0).OrderByDescending(p => p.Stock_Qty).First();
D) products.Aggregate((max, next) => next.Stock_Qty > max.Stock_Qty ? next : max);
E) products.Select(p => new { p.Prod_Name, p.Stock_Qty }).OrderByDescending(p => p.Stock_Qty).First();
Cevap: B
65. En fazla fatura oluşturan müşteriyi bulmak için doğru ifade hangisidir?
A) invoices.Select(i => i.Cust_ID).GroupBy(c => c).OrderByDescending(g => g.Count()).First();
B) customers.Join(invoices, c => c.Cust_ID, i => i.Cust_ID, (c, i) => i.Cust_ID).GroupBy(id => id).OrderByDescending(g => g.Count()).First().Key;
C) invoices.GroupBy(i => i.Cust_ID).OrderByDescending(g => g.Count()).First().Key;
D) customers.GroupJoin(invoices, c => c.Cust_ID, i => i.Cust_ID, (c, i) => new { c.Cust_ID, Count = i.Count() }).OrderByDescending(c => c.Count).First();
E) invoices.GroupBy(i => i.Cust_ID).Select(g => g.Key).Max();
Cevap: C

66. 300$ üzerindeki faturaların toplamını bulmak için doğru ifade hangisidir?
A) invoices.Sum(i => i.Invoice_Total > 300 ? i.Invoice_Total : 0);
B) invoices.Where(i => i.Invoice_Total > 300).Sum(i => i.Invoice_Total);
C) invoices.Select(i => i.Invoice_Total).Where(t => t > 300).Sum();
D) invoices.Aggregate(0.0, (sum, i) => i.Invoice_Total > 300 ? sum + i.Invoice_Total : sum);
E) invoices.Where(i => i.Invoice_Total >= 300).Sum(i => i.Invoice_Total);
Cevap: B

67. En pahalı ürünün adını bulmak için doğru ifade hangisidir?
A) products.Where(p => p.Prod_Price == products.Max(p => p.Prod_Price)).First().Prod_Name;
B) products.Max(p => p.Prod_Price).Select(p => p.Prod_Name);
C) products.Aggregate((max, next) => next.Prod_Price > max.Prod_Price ? next : max).Prod_Name;
D) products.OrderByDescending(p => p.Prod_Price).First().Prod_Name;
E) products.OrderByDescending(p => p.Prod_Price).Select(p => p.Prod_Name).First();
Cevap: D

68. 2023 yılı içinde oluşturulan faturaları filtrelemek için doğru ifade hangisidir?
A) invoices.Where(i => i.Invoice_Date.Year == 2023).ToList();
B) invoices.Select(i => new { i.Invoice_ID, i.Invoice_Date }).Where(i => i.Invoice_Date.Year == 2023);

C) invoices.Where(i => i.Invoice_Date >= new DateTime(2023, 1, 1) && i.Invoice_Date <= new DateTime(2023, 12, 31));
D) invoices.Where(i => i.Invoice_Date.Year.Equals(2023)).ToList();
E) invoices.Where(i => i.Invoice_Date.ToString("yyyy") == "2023").ToList();
Cevap: C

69. Stokta kalmayan ürünleri listelemek için doğru ifade hangisidir?
A) products.Where(p => p.Stock_Qty == 0).ToList();
B) products.Select(p => new { p.Prod_Name, p.Stock_Qty }).Where(p => p.Stock_Qty == 0);
C) products.Where(p => p.Stock_Qty <= 0).ToList();
D) products.Where(p => p.Stock_Qty.Equals(0)).Select(p => p.Prod_Name);
E) products.Where(p => p.Stock_Qty < 1).ToList();
Cevap: A

70. Fatura tarihine göre sıralı ve müşteri adına göre grup düzeninde bir liste oluşturmak için doğru ifade hangisidir?
A) invoices.Select(i => new { i.Invoice_Date, i.Cust_ID }).GroupBy(i => i.Cust_ID).OrderBy(i => i.Invoice_Date);
B) customers.Join(invoices, c => c.Cust_ID, i => i.Cust_ID, (c, i) => new { c.Cust_Name, i.Invoice_Date }).OrderBy(i => i.Invoice_Date);
C) invoices.GroupBy(i => i.Cust_ID).OrderBy(i => i.Invoice_Date);
D) invoices.OrderByDescending(i => i.Invoice_Date).GroupBy(i => i.Cust_ID);
E) invoices.OrderBy(i => i.Invoice_Date).GroupBy(i => i.Cust_ID);
Cevap: E